



Flux Technical Assessment

Prepared for: Data Engineer (Intermediate)

Estimated time: 2-3 hours

Due: 3 working days from receipt of assessment

1. Introduction

Thank you for your interest in the Data Engineer role at Flux Full Circle. This practical assessment reflects the day-to-day reality of the role at an intermediate level, not just cleaning a single file, but building a re-runnable pipeline that ingests incremental loads, models data correctly for reporting, and stands up to production concerns of performance, cost and testing.

We are interested in engineering judgement as much as a working result; how you model the data, how you make the pipeline safe to re-run, how you reason about scale, and how you prove your logic with tests. If you run short on the 2-3 hours, prioritise correctness and tests over completeness, and note what you would do with more time.

2. What to expect

Item	Detail
Time	Designed to take no more than 2-3 hours. Please do not spend longer, if you run short, note what you would have done with more time.
Tools	Python 3 and any SQL environment. BigQuery is our production warehouse; a free BigQuery sandbox, DuckDB, or Postgres are all fine. Please write SQL in a BigQuery-compatible dialect where practical.
Data	Four CSV files (see section 3). No Flux systems, credentials or client data are involved.
AI tools	You may use AI assistants (e.g. Copilot, ChatGPT, Claude). If you do, tell us where and how. We value honest, effective tool use, however you will need to be able to explain and defend your code if asked in the interview.
Submission	A Git repository (preferred) or a ZIP. See section 4.



3. The scenario

Flux consolidates booking data from a portfolio of luxury lodges and hotels into a BigQuery warehouse that powers client revenue reporting. The source system sends data as daily batches: a first full extract, then daily incremental deltas containing new bookings and updates to existing ones. Bookings can be amended (guest counts corrected, revenue restated) or cancelled after they are first created.

Your task is to build a pipeline that turns these batches into a trustworthy, well-modelled, analysis-ready dataset that is safe to re-run, and to write the SQL that models and reports on it.

Files provided:

1. Bookings_batch1_2025-09-01.csv. The initial full load (messy, as real extracts are).
2. Bookings_batch2_2025-09-02.csv. The next day's incremental delta: new bookings, updates to existing bookings, and some re-sent rows.
3. Properties.csv. The property dimension.
4. Fx_rates.csv. The currency rates to ZAR, versioned by effective date (see the point-in-time note below).

Column	Description
booking_id	Business key. The same booking_id can appear more than once, across and within batches.
created_at / updated_at	When the booking was created and last amended. updated_at tells you which version is newest.
booking_status	confirmed, pending or cancelled. Amendments can change status.
check_in / check_out	Mixed date formats. room_rate/revenue/currency/booking_channel carry the usual free-text inconsistencies.

Point-in-time FX: fx_rates.csv gives each currency several rates, each with an effective_from date. Convert a booking's revenue to ZAR using the rate that was effective on the booking's created_at date, not a single latest rate.



4. What to submit

1. Your pipeline project (code, config, tests) and instructions to run it.
2. Your SQL (.sql).
3. The output artefacts: analysis-ready table, rejects, data-quality summary and merge audit.
4. A short README (assumptions, how to run, test command, and where you used AI).
5. Your Part C write-up.

Submit a Git repository (this is preferred as we value seeing your commit history) or a single ZIP. We will ask you to walk us through your reasoning and code at the interview stage.

5. How we assess

We score across five areas. The relative weighting reflects what matters most in the role.

Area	Weight	In short
Incremental pipeline & idempotency	22%	Correct merge, safe re-runs, stale/re-send handling
Data modelling & reporting views	20%	Sound model, partition/cluster, reusable views, status logic
Testing & engineering rigour	20%	Real unit tests, config, structure, re-runnable
Performance & cost at scale	16%	Vectorised, set-based, BigQuery cost awareness
SQL analytics	14%	Point-in-time FX, window functions, DQ checks
Communication (Part C)	8%	Clear productionisation and trade-offs

Please see Parts A, B and C of the assessment below. Good luck!



PART A: SQL & Modelling

Write BigQuery-compatible SQL. Assume the cleaned bookings are available as a table. Provide your SQL in a .sql file with brief comments.

1. Deduplicate to the current version of each booking. One row per booking_id, keeping the most recent by updated_at (a window function/QUALIFY is the idiomatic approach).
2. Point-in-time revenue in ZAR. Join each booking to the FX rate effective on its created_at date, and expose revenue_zar.
3. A reporting model - DDL for a fact table partitioned and clustered appropriately for our common queries (monthly trends, per-property, per-channel), plus one reusable reporting view the analytics team could build on. Explain your partition/cluster choices in a comment.
4. Month-on-month revenue growth for confirmed bookings in 2025 (ZAR), showing each month's revenue and the % change on the prior month (window function).
5. Top 2 properties by revenue within each country (ZAR), handling ties sensibly.
6. Data-quality assertions as SQL. One row per check with a count (e.g. duplicate current-version ids, orphan properties, non-positive/missing revenue, invalid date ranges, bookings with no applicable FX rate). Run these against the raw data so they actually surface counts.

NOTE: Treat cancelled bookings deliberately in your reporting queries and state whether you included or excluded them and why.

PART B: Incremental Pipeline

Build a re-runnable pipeline (a small, well-structured Python project) that:

1. Ingests both batches and merges them into a single current-state table: new bookings inserted, updates applied, keeping the latest version per booking_id by updated_at.
2. Is idempotent: re-running it (or re-processing batch 2) must not create duplicates or change the result. Handle re-sent rows and stale/older updates that arrive out of order. An older update must not overwrite a newer value.



3. Cleans and standardises dates, currencies, channels, whitespace and numerics, and enriches with point-in-time FX to ZAR and property attributes.
4. Models status: make a clear, documented decision on how cancelled and pending bookings are represented in the analysis-ready output.
5. Validates and quarantines bad records with reasons; emits an analysis-ready table, a rejects table, a data-quality summary, and a merge audit (rows inserted/updated/unchanged).
6. Is engineered to standard: clear module structure, configuration for paths/thresholds, logging, and vectorised transforms (avoid row-by-row loops). Prefer set-based operations that would remain reliable at scale.

NOTE: Automated tests are required, not optional. Include unit tests (e.g. pytest) covering at least your merge/idempotency logic, the point-in-time FX join, and a couple of cleaning/validation rules.

PART C: Production write-up

In no more than one page, cover how you would run this for real on GCP and BigQuery:

1. Incremental loading into BigQuery (e.g. MERGE/upsert), partitioning and clustering, and how you would keep query cost down as the data grows to tens of millions of rows.
2. Orchestration, scheduling, and how you would monitor pipeline health and catch data-quality regressions before they reach reporting.
3. Version control, testing and CI/CD, plus documentation/SOPs and lineage for long-term maintainability.
4. Key assumptions and trade-offs given the time limit. A note on how you would approach RAG/AI enrichment of this data is welcome but optional.